Sparsity-Aware Winograd CNN Acceleration with Zero-Row Skipping and Clock Gating

JunBeom Park[1], Taewook Kang[2]
[1]School of Electronic and Electrical Engineering
[2]Department of Semiconductor Convergence Engineering
Sungkyunkwan University, Suwon, Korea
[1]jbpark020616@skku.edu, [2]twkang@g.skku.edu

I. INTRODUCTION

Activation sparsity enables redundant computations to be eliminated in convolutional neural networks (CNNs) [1]. Stacked 3×3 convolutions enlarge receptive fields with additional nonlinearities [2]. Winograd F(2×2,3×3) reduces multiplications for four outputs from 36 to 16 [3], but does not necessarily preserve spatial-domain sparsity [4].

Prior work applies activation functions and pruning in the Winograd domain [4]. We exploit intermediate zero rows through schedule compaction and integrated clock gating (ICG). Single-core experiments isolate their effects; the architecture extends to 16 cores. Controlled experiments characterize sensitivity to operand supply.

II. PROPOSED ARCHITECTURE

Winograd F(2×2,3×3) computes a 2×2 output tile from a 4×4 input tile and a 3×3 filter. With transformed inputs V and filters U, the output for channel k is

$$Y_k = A^T \left(\sum_c U_{k,c} \odot V_c\right) A, \quad V_c = B^T d_c B, \quad U_{k,c} = G g_{k,c} G^T$$

where d and g are the input tile and filter; A, B, and G are transform matrices; c indexes input channels; and $\odot$ denotes element-wise multiplication. Since V = TB transforms each row of T = B^Td independently, an all-zero row remains zero. A reduction-NOR detects these rows in intermediate registers (Fig. 1). The mask is retained with V and reused across output channels to skip the corresponding multiply–accumulate (MAC) operations without changing the output.

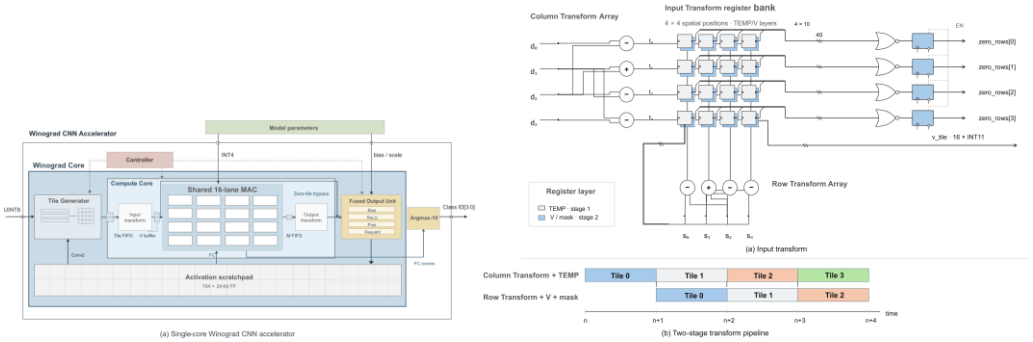


Fig. 1. Single-core architecture and input-transform zero-row detection.

The scheduler skips masked rows and issues remaining unmasked rows consecutively (Fig. 2, left). Schedule compaction preserves weight alignment and accumulation order, reducing cycles when operands are available. Supply stalls limit the reduction. ICG suppresses unnecessary clock activity. All-zero work items bypass computation while preserving output ordering, bias addition, and post-processing.

The 16-core architecture applies the same zero-row skipping mechanism (Fig. 2, right). Output-channel partitioning processes shared activations with different filters in parallel. Multicast distributes activations; static random-access memory (SRAM) supports transformed-data reuse and per-core weight delivery.

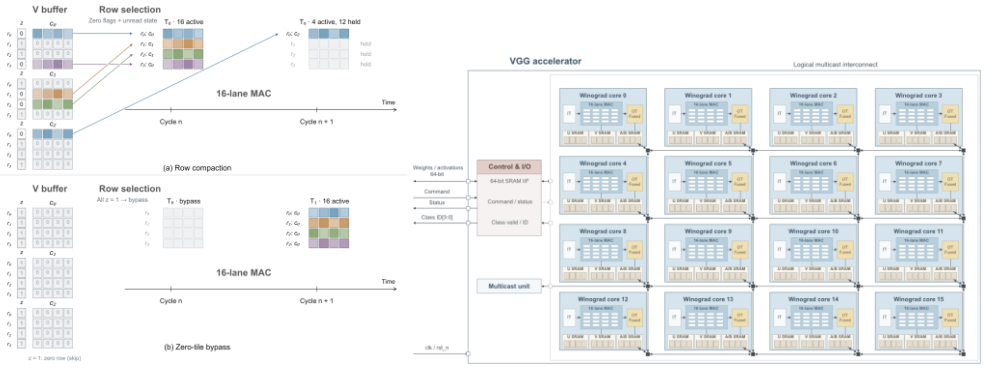


Fig. 2. Left: row selection and schedule compaction. Right: the 16-core architecture.

ACKNOWLEDGMENT
This work was supported by Korea Institute for Advancement of Technology(KIAT) grant funded by the Korea Government(Ministry of Education) (P0025675, “Semiconductor”-Specialized University).

III. EXPERIMENTAL RESULTS

We evaluated an MNIST CNN and VGG-16-BN adapted for CIFAR-10 using Yosys and OpenROAD with the ASAP7 predictive 7-nm library [5]. Results are post-synthesis estimates before placement and routing. Power is estimated from zero-delay gate-level simulation value change dump (VCD) traces; energy equals average power multiplied by duration. Inputs and clocks are matched within each comparison.

Table I isolates the effects of zero-row skipping and ICG. Accuracy was 97.0% on 100 distinct MNIST images repeated ten times. Their combined application reduced execution cycles by 7.11% and energy by 55.90%; skipping provided a further 15.02% energy reduction over ICG alone. The 0.46% net area reduction includes synthesis-mapping effects.

TABLE I. INDIVIDUAL AND COMBINED EFFECTS OF ZERO-ROW SKIPPING AND ICG

Configuration	**Mean cycles/image**	**Area (μm²)**	**Power (mW)**	**Energy (nJ/image)**
Common architecture	1,072.00	7,804.64	16.5916	17.8861
Zero-row skipping	995.83	7,781.48	16.2739	16.3133
ICG	1,072.00	7,802.57	8.6100	9.2818
Skipping + ICG	995.83	7,769.03	7.8685	7.8876

ASAP7 RVT FF, 0.77 V, 0°C, 1 GHz. Cycles: 100 distinct images. Power/energy: 20 images including initialization.

Conv1 receives one pixel per cycle externally, limiting cycle reductions; Conv2 uses the internal scratchpad and benefits more directly from skipping. For one VGG input (Fig. 3, left), the convolution MAC count decreased from 313.20M direct MACs to 139.20M with Winograd and 96.38M with zero-row skipping, a further 30.76% reduction.

The 16-core design occupied 742,748.20 μm² including SRAM, with a post-synthesis minimum period of 1,377.62 ps (725.89 MHz). A fixed-mapping core ablation estimated a 0.58% SRAM-inclusive area overhead for zero-row skipping.

Controlled core-level experiments used a 4×4×32 input and eight output channels with resident weights. We varied the zero-row fraction and 64-bit input-word interval, comparing execution with zero-row skipping enabled and disabled on the same netlist with ICG enabled (Fig. 3, right).

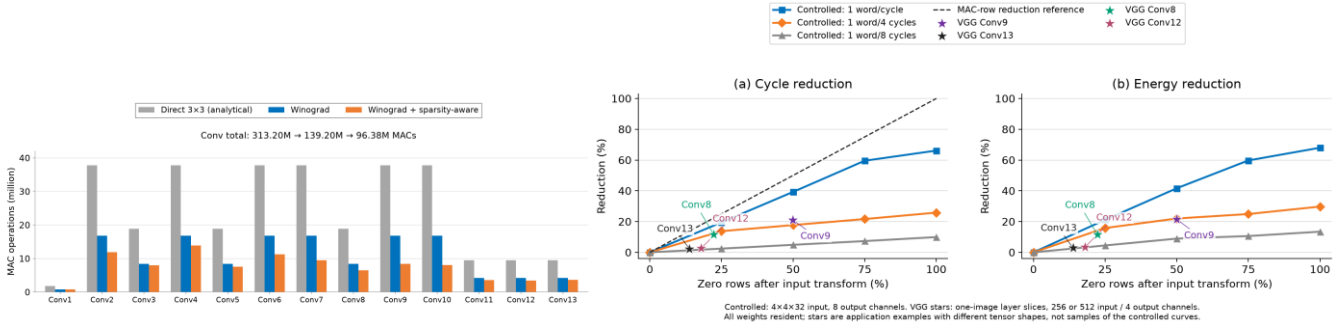


Fig. 3. Left: VGG MAC counts (direct: analytical, including padding; Winograd: RTL-issued MAC-lane operations), excluding transforms and fully connected layers. Right: cycle and energy reductions with eight SRAM macros, a 1.43-ns clock, and resident weights. Energy spans command start to completion, excluding external activation SRAM and multicast. Transfers follow the ready handshake; stars denote VGG layer examples with different tensor shapes and four output channels.

At 50% zero rows, one word/cycle reduced cycles and energy by 39.28% and 41.64%, versus 4.89% and 8.92% at one word/eight cycles. Operand-supply stalls thus limit schedule-compaction benefits. VGG examples use one CIFAR-10 image: Conv9 exhibited 49.93% zero rows, with cycle and energy reductions of 21.08% and 21.51%. These examples use tensor shapes different from the controlled workload.

IV. CONCLUSION

Zero-row detection, schedule compaction, and ICG reduced single-core energy by 55.90%, with a 15.02% energy reduction over ICG alone. The architecture extends to 16 cores through output-channel partitioning. Controlled experiments show that operand supply limits achievable cycle and energy reductions. Future work targets SRAM access, multicast, weight delivery, and tensor partitioning.

REFERENCES

[1] A. Parashar et al., “SCNN: An accelerator for compressed-sparse convolutional neural networks,” in Proc. ISCA, 2017, pp. 27–40.

[2] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” in Proc. ICLR, 2015.

[3] A. Lavin and S. Gray, “Fast algorithms for convolutional neural networks,” in Proc. CVPR, 2016, pp. 4013–4021.

[4] X. Liu, J. Pool, S. Han, and W. J. Dally, “Efficient sparse-Winograd convolutional neural networks,” in Proc. ICLR, 2018.

[5] L. T. Clark et al., “ASAP7: A 7-nm finFET predictive process design kit,” Microelectronics Journal, vol. 53, pp. 105–115, 2016, doi: 10.1016/j.mejo.2016.04.006.